

Kode C# ini adalah sebuah fungsi untuk melakukan **Image Negation** (membuat efek negatif/klise foto) pada sebuah gambar digital. Kode ini dirancang untuk **performa tinggi** karena menggunakan teknik manipulasi memori langsung (*Direct Memory Access*) menggunakan LockBits, bukan menggunakan metode GetPixel dan SetPixel yang lambat.

Berikut adalah penjelasan detail baris per baris:

---

## 1. Header dan Namespace

C#

```
using System.Drawing.Drawing2D;  
using System.Drawing.Imaging;  
using System.Runtime.InteropServices;
```

- **System.Drawing.Drawing2D:** Menyediakan fungsi grafis tingkat lanjut.
- **System.Drawing.Imaging:** Sangat penting di sini. Namespace ini menyediakan kelas BitmapData dan enumerasi ImageLockMode yang digunakan untuk memanipulasi data piksel secara langsung.
- **System.Runtime.InteropServices:** Menyediakan kelas Marshal. Ini jembatan antara kode C# yang *managed* (dikelola otomatis oleh .NET) dan memori *unmanaged* (memori mentah sistem). Kita butuh ini untuk menyalin data piksel.

---

## 2. Definisi Kelas dan Metode

C#

```
namespace MiniPhotoshop.Logic.ImageProcessing  
{  
    public static class ImageNegation  
    {  
        public static Bitmap Apply(Bitmap currentImage)  
        {
```

- **namespace ...:** Mengelompokkan kode agar rapi.

- **public static class:** Kelas utilitas statis, artinya kita tidak perlu membuat objek new ImageNegation() untuk menggunakannya.
  - **public static Bitmap Apply(...):** Metode ini menerima gambar input (currentImage) dan akan mengembalikan gambar baru (Bitmap) yang sudah dinegasikan.
- 

### 3. Persiapan Gambar (Cloning)

C#

```
Bitmap resultImage = new Bitmap(currentImage);
```

- Kita membuat salinan (clone) dari gambar asli.
  - **Mengapa?** Agar gambar asli (currentImage) tidak rusak atau berubah. Kita hanya memodifikasi salinannya (resultImage).
- 

### 4. Mengunci Memori (LockBits)

C#

```
Rectangle rect = new Rectangle(0, 0, resultImage.Width, resultImage.Height);  
BitmapData bmpData = resultImage.LockBits(rect, ImageLockMode.ReadWrite,  
resultImage.PixelFormat);
```

- **Rectangle rect:** Kita menentukan area mana dari gambar yang ingin kita edit. Di sini kita memilih **seluruh gambar** (dari 0,0 sampai lebar dan tinggi penuh).
  - **LockBits:** Ini adalah **inti dari performa kode ini**.
    - Komputer mengunci area memori tempat gambar disimpan agar tidak dipindahkan oleh *Garbage Collector* atau diakses proses lain saat kita sedang mengeditnya.
    - Fungsi ini mengembalikan objek BitmapData yang berisi informasi detail tentang lokasi memori piksel.
    - ImageLockMode.ReadWrite: Kita memberitahu sistem bahwa kita akan membaca DAN menulis data piksel tersebut.
- 

### 5. Menyalin Data ke Array Byte

C#

```
IntPtr ptr = bmpData.Scan0;  
int bytes = Math.Abs(bmpData.Stride) * resultImage.Height;  
byte[] rgbValues = new byte[bytes];
```

```
Marshal.Copy(ptr, rgbValues, 0, bytes);
```

- **ptr = bmpData.Scan0**: Scan0 adalah **pointer** (alamat memori) ke piksel pertama (kiri atas) pada gambar.
  - **int bytes**: Menghitung total jumlah byte dalam gambar.
    - **Penting**: Kita menggunakan Stride, bukan Width. Stride adalah lebar baris data dalam memori (yang seringkali ditambah *padding* agar pas dengan arsitektur processor), sedangkan Width hanyalah lebar piksel visual. Rumusnya: Stride \* Height.
  - **byte[] rgbValues**: Kita membuat array kosong di C# untuk menampung data piksel tersebut.
  - **Marshal.Copy(...)**: Kita menyalin data dari memori mentah (ptr) ke dalam array C# kita (rgbValues). Sekarang kita bisa mengedit array ini dengan sangat cepat.
- 

## 6. Persiapan Loop

C#

```
int bytesPerPixel = Image.GetPixelFormatSize(resultImage.PixelFormat) / 8;  
int stride = bmpData.Stride;
```

- **bytesPerPixel**: Menentukan kedalaman warna.
    - Jika formatnya 32bpp (bits per pixel), maka 1 piksel = 4 byte (Red, Green, Blue, Alpha).
    - Jika 24bpp, 1 piksel = 3 byte (RGB).
    - Dibagi 8 karena 1 byte = 8 bit.
  - **stride**: Menyimpan nilai stride ke variabel lokal untuk akses cepat di dalam loop.
- 

## 7. Loop Pemrosesan (Image Processing)

C#

```
for (int y = 0; y < resultImage.Height; y++)  
{  
    int rowOffset = y * stride;  
    for (int x = 0; x < resultImage.Width; x++)  
    {
```

- Kita melakukan perulangan (looping) untuk setiap baris (y) dan setiap kolom (x) piksel.
- **rowOffset = y \* stride**: Menghitung posisi awal memori untuk baris y saat ini.

C#

```
int i = rowOffset + (x * bytesPerPixel);
```

- **int i**: Ini adalah indeks array satu dimensi untuk piksel di koordinat (x, y).
- Rumusnya memetakan koordinat 2D (x,y) ke array linear 1D.

C#

```
    rgbValues[i] = (byte)(255 - rgbValues[i]);  
    rgbValues[i + 1] = (byte)(255 - rgbValues[i + 1]);  
    rgbValues[i + 2] = (byte)(255 - rgbValues[i + 2]);  
}  
}
```

- Inilah logika **Negasi (Invert)** warnanya.
- Dalam format bitmap Windows, urutan byte biasanya adalah **B G R** (Blue, Green, Red).
  - `rgbValues[i]` : Channel Blue
  - `rgbValues[i + 1]` : Channel Green
  - `rgbValues[i + 2]` : Channel Red
- **Logika Matematika**:
  - Nilai warna berkisar 0 (gelap) sampai 255 (terang).

- 255 - nilai asli akan membalik warnanya.
- Contoh: Jika pixel putih (255), maka  $255 - 255 = 0$  (hitam). Jika pixel hitam (0), maka  $255 - 0 = 255$  (putih).
- **(byte)**: Hasil pengurangan harus di-cast kembali menjadi tipe byte.

---

## 8. Menyimpan Kembali dan Membersihkan Memori

C#

```

        Marshal.Copy(rgbValues, 0, ptr, bytes);
        resultImage.UnlockBits(bmpData);

    return resultImage;
}
}
}

```

- **Marshal.Copy(...)**: Setelah array rgbValues selesai dimodifikasi di dalam loop, kita **menyalin balik** data dari array tersebut ke alamat memori gambar (ptr). Ini langkah yang memperbarui gambar sesungguhnya.
- **resultImage.UnlockBits(bmpData)**: **Sangat Penting!** Kita harus melepas kunci memori. Jika tidak, gambar tersebut akan terkunci oleh sistem operasi dan bisa menyebabkan error atau kebocoran memori (memory leak).
- **return resultImage**: Mengembalikan gambar hasil editan ke pemanggil fungsi.

### Ringkasan Alur Kerja

1. **Clone** gambar asli.
2. **Kunci (Lock)** memori gambar agar aman diakses.
3. **Salin** data piksel dari memori sistem ke array C#.
4. **Looping** setiap piksel dan lakukan matematika  $255 - \text{nilai}$  untuk membalik warna.
5. **Salin balik** array yang sudah diedit ke memori sistem.
6. **Buka Kunci (Unlock)** memori.

Apakah Anda ingin saya menjelaskan bagaimana cara memodifikasi kode ini untuk membuat efek Grayscale (hitam putih) atau Sepia?